

WHITE PAPER

Encrypted Zero-Knowledge for Private Peer-to-Peer Payments on Stacks

John B Mukhwana

September 2026

Reference implementation	Stacks Shield (proof of concept)
Network	Stacks Testnet
Deployer (v2, current)	ST18XMPE0PS5VNEEK82BPW7NRZRHXEPH16JK8NN6
Deployer (v1, superseded)	ST2HXRZ8A82JJAP14KD83JEXNRCF34J67088WJSJH

Abstract

Every balance and every payment on a public blockchain is visible to anyone. On Stacks that means salaries, treasuries, trades, and ordinary personal payments are all in the open. This paper presents a protocol that lets people hold and move value privately on Stacks. Instead of transparent balances, users hold shielded notes and move value with zero-knowledge proofs. One shared pool covers native STX and SIP-10 tokens such as sBTC and USDCx, so privacy is not fragmented across assets. A user deposits a public amount into a private note, then transfers, splits, merges, and withdraws value without revealing amounts or participants, and a relayer keeps the user's address off chain.

Stacks contracts cannot yet verify this kind of proof on chain at an acceptable cost, so verification is delegated to zkVerify, an external network that checks each proof and batches many into a single commitment. The Stacks contracts then perform a cheap inclusion check. This paper explains the problem, the design, the privacy and trust model, the fee economics, and the roadmap toward removing the external dependency. The formal construction, with all equations and circuit relations, is in the companion Yellow Paper.

The protocol runs today as a working proof of concept called Stacks Shield, deployed on Stacks testnet, which validates the full design end to end for STX, sBTC, and USDCx.

Contents

1	The Problem	3
2	The Idea	3
3	How It Works	4
3.1	The proof pipeline	4
4	One Pool for Many Assets	5
5	What Is Private and What Is Public	5
6	Fees and Economics	5
7	Use Cases	6
8	Trust and Security	6
9	Roadmap	6
10	Conclusion	7
	References and Further Reading	7

1 The Problem

A public blockchain is a public ledger. On Stacks, anyone can look up how much STX or how many SIP-10 tokens an address holds, and can follow every transfer that address has ever made. There is no setting that turns this off. The ledger is the product.

For many real uses that is a dealbreaker.

- A company that pays salaries on chain exposes every employee's pay, and exposes its own payroll to competitors.
- A treasury that holds funds on chain broadcasts its size, its runway, and every move it makes.
- A trader who moves size reveals their positions and invites front-running.
- An ordinary person who receives sBTC or a stablecoin hands their full financial history to anyone who learns one address.

Privacy is not a niche feature for these users. It is the difference between using a chain and not using it.

2 The Idea

The protocol replaces visible balances with private notes. A note is a record of value that only its owner can read and only its owner can spend. On chain, a note appears as a commitment, which is a cryptographic fingerprint that hides the amount, the owner, and everything else about it.

Users move value between notes by publishing a zero-knowledge proof. The proof convinces the network that an operation follows every rule, that the spender owns the input, that no value is created, that the note has not already been spent, without revealing the amount or the people involved. The chain learns that the move was valid. It learns nothing else.

Five operations cover the full lifecycle.

Operation	What it does
Shield	Turn a public amount into a private note (a deposit).
Transfer	Send a note to someone else, privately.
Split	Divide one note into two.
Merge	Combine two notes into one.
Withdraw	Turn a note back into a public amount at any address.

Shield and withdraw sit at the edge between the public world and the private pool, so their amounts are visible at that moment. Everything in between is hidden.

3 How It Works

The design rests on four ideas. This section explains each in plain terms. The exact constructions are specified in the Yellow Paper.

Notes and commitments. A note holds an amount, an owner key, and a random blinding value. The chain stores only a commitment to the note, a hash that reveals nothing. Two notes of the same amount look completely different on chain, because each has its own random blinding.

One shared tree. Every commitment, for every asset, is added to a single Merkle tree. The tree has one short root that stands in for the whole set. To spend a note, a user proves that its commitment is in the tree, without revealing which one it is.

Nullifiers stop double-spends. When a note is spent, the operation publishes a nullifier, a one-time tag derived from the note's secret. The network records every nullifier and rejects any repeat. Because the tag is derived from hidden data, no observer can connect it to the note it retired.

A relayer hides the sender. A zero-knowledge proof hides what moved, but a transaction still has to be sent by someone, and that sender is public. To close the gap, private operations are submitted by a relayer. The transaction arrives from the relayer's address, so the user never appears on chain. The relayer cannot change the operation, because every detail is fixed inside the proof. It can only choose to send it or not.

3.1 The proof pipeline

A proof is produced on the user's device, checked by an external verification network called zkVerify, and then accepted on Stacks through a cheap inclusion check. Figure 1 shows the path.

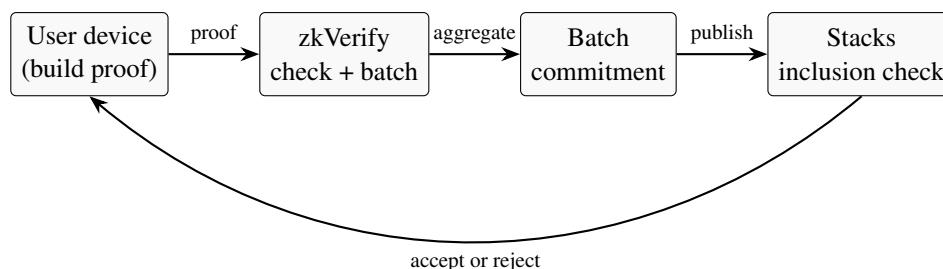


Figure 1: From proof to settlement. Stacks accepts an operation only when its proof was checked by zkVerify and included in a published batch.

Why the detour through zkVerify? Verifying this kind of proof directly inside a Stacks contract is not yet practical, because Clarity, the Stacks contract language, lacks the cryptographic operations it would need. Delegating the check to zkVerify keeps the on-chain cost low today. Section 9 describes how that dependency is removed over time.

4 One Pool for Many Assets

Privacy depends on the crowd. If each asset had its own pool, each crowd would be smaller and easier to analyze. The protocol uses one shared pool for STX and for every supported SIP-10 token, including sBTC and USDCx, so they all hide in the same set.

The assets stay separated where it matters. Each note is bound to its asset, so a note of one token can never be spent as another. The pool tracks each asset independently and can never pay out more of an asset than was deposited. A new token is added by a single on-chain registration, with no change to the apps or the contracts.

The result is one anonymity set that grows with total usage across every asset, while each asset's accounting stays exact and isolated.

5 What Is Private and What Is Public

Here is the exact boundary.

Hidden. Inside the pool, amounts, owners, and the links between them are hidden. A transfer, split, or merge reveals none of the amounts and none of the participants. Note contents are encrypted so that only the owner can read them.

Public. Shielding and withdrawing touch the transparent world, so the amount and the address involved at that moment are visible, like any normal deposit or withdrawal. The number of operations over time is visible. The fact that the pool exists and how much it holds in total is visible.

Practical privacy. Strong cryptography is necessary but not sufficient. Privacy in practice also depends on how the pool is used. It is strongest when the pool is busy, when amounts use common denominations, and when there is time between a deposit and a later withdrawal. A large, unusual amount that goes in and comes straight back out is easier to correlate no matter how good the proofs are.

6 Fees and Economics

The protocol has no token. There is nothing to buy, stake, or speculate on to use it. The only economics is a small fee that keeps the protocol maintainable, and that fee is designed to preserve privacy.

Operation	Fee	How it is charged
Shield	0.25%	folded into the public deposit
Withdrawal	0.30%	taken from the payout by the pool itself
Transfer / Split / Merge	none	no percentage is possible over hidden amounts

Only shield and withdrawal expose a public amount, so only they can carry a percentage fee. The private operations move hidden amounts, so the protocol cannot take a percentage there and does not charge one. The withdrawal fee is paid out by the pool contract rather than by the user directly, so a fee payment cannot be linked back to a person. Fees are configured per asset and are paid by the user, never by the relayer.

7 Use Cases

- **Payroll and vendor payments.** A company shields a budget once, then pays salaries and invoices privately, without publishing each amount or each recipient's balance.
- **Treasury operations.** An organization can hold and move funds without broadcasting its size or telegraphing every action.
- **Trading and OTC settlement.** Counterparties can settle without revealing position size to the whole market.
- **Private savings and payments for individuals.** A person can receive sBTC or a stablecoin and spend it without exposing their full history to anyone who learns one address.
- **Bitcoin-native privacy.** Because sBTC is a first-class asset in the pool, the protocol brings practical payment privacy to Bitcoin value on Stacks.

8 Trust and Security

The protocol is designed so that no operator can move a user's funds and no operator can forge a valid operation. A few dependencies are stated plainly here.

- **Delegated verification.** Proofs are checked by zkVerify, not by Stacks consensus. This is a real trust and liveness dependency today. Section 9 sets out how it is removed.
- **Root publication.** The result of that off-chain check is posted to Stacks by an authorized publisher. Today that publisher is trusted. The protocol runs it through a small dedicated set rather than a single admin key, and the planned fix requires several independent operators to co-sign each posting. Until then, this posting is not trustless.
- **Sender privacy relies on the relayer.** The relayer cannot alter or steal an operation. It can refuse to send one. Spreading the relayer role across independent operators removes that as a chokepoint.
- **Not audited.** The proof of concept, Stacks Shield, is on testnet and has not had an independent third-party audit. An internal security review has been completed and is published in the project repository. An external audit is required before any mainnet deployment.

9 Roadmap

The direction of travel is to remove every external dependency until the protocol stands on Stacks alone. The steps, in order of feasibility:

1. **Independent verification, off chain.** Move the proof check in-house across several independent operators who each verify and co-sign the result. The contract checks their signatures with primitives Clarity already has. This decentralizes today's single dependency without any circuit changes.

2. **Native verification on chain.** As Stacks gains the cryptographic operations it currently lacks, the proof check moves into the contracts themselves, and the external network is no longer needed. There is more than one technical route to this, and the design keeps the door open to each.
3. **A decentralized relay set.** Independent relayers remove the last liveness chokepoint for sender privacy.

Together these turn a protocol that leans on external services today into one that lives entirely on Stacks.

10 Conclusion

Public ledgers make privacy the exception when it should be the default. This protocol gives Stacks users a practical way to hold and move value privately, across STX and the tokens people actually use, with cryptography enforcing the rules and a relay removing the last public link. The open problem is trust-minimized verification, and the path to solving it is concrete. The delegation to zkVerify is a deliberate first step with a clear plan to remove it.

For the full construction, including every definition, equation, and circuit relation, see the companion Yellow Paper.

References and Further Reading

- [1] Companion Yellow Paper: the formal specification of this protocol.
- [2] E. Ben-Sasson et al. Zerocash: Decentralized Anonymous Payments from Bitcoin. The Zcash protocol specification.
- [3] Aztec. Barretenberg and the UltraHonk proof system.
- [4] zkVerify. Proof verification and aggregation network. <https://zkverify.io>
- [5] Stacks and Clarity documentation. <https://docs.stacks.co>
- [6] Protocol security review (internal). See `audits/` in the project repository.